CatEatFish AI (แมวกินปลา AI) — Capstone Project Report

Project Title: CatEatFish AI — Intelligent Thai Sea Fish Culinary Guide & Cold-Chain Fishery Assistant

Data Source: https://cateatfish.co/sea/thai-fish (CAT EAT FISH — ประมงพื้นบ้านบ้านน้ำเค็ม จ.พังงา)

Model Used: `gemma-4-E4B-it` via Course OpenAI-Compatible Proxy (`https://llm.nat-d.uk/v1`)

---

1. Problem Statement & Motivation

Thailand's Andaman Sea and Gulf of Thailand host an extraordinary diversity of edible local sea fish—over 40 species caught sustainably by small-scale artisanal fishers (such as the Baan Nam Khem community and Moken sea nomads in Phang Nga province). However, most Thai consumers only recognize a handful of commercial fish (such as sea bass or salmon) and do not know how to cook unfamiliar local species like **ปลากะมงพร้าว (Giant Trevally)**, **ปลาแซกำ (Yellowspotted Trevally)**, **ปลาเหลืองญี่ปุ่น**, **ปลาอังเกย (Golden Snapper)**, **ปลาจ้าวสมุทร**, or **ปลาเก๋าดอกดำ (Black-tipped Grouper)**. Worse still, cooking a local fish the wrong way—such as deep-frying thick-skinned groupers (`ปลาเก๋าดอกดำ` , `ปลาเก๋าเพลิง`) or high-fat belly fish (`ปลาจ้าวสมุทร`)—ruins the dish.

CatEatFish AI (แมวกินปลา AI) solves this real-world problem by pairing:

1. **Unstructured Culinary & Fishery Knowledge Base (Hybrid RAG):** 5 structured Markdown guides (15 chunks) curated directly from `https://cateatfish.co/sea/thai-fish` , covering seasonal availability, meat texture, recommended dishes (Sashimi, Fish & Chips, steamed, spicy sour curry, kid/elderly-friendly meals), cooking warnings, sustainable fishing gear (`อวนลอย` , `ลอบดักปลา`), and `-20°C` cold-chain standards.

2. **Structured Relational Store Database (Guarded Text-to-SQL):** A 3-table SQLite database (`products` , `orders` , `returns`) tracking 15 vacuum-sealed seafood SKUs, 20 cold-chain customer orders (`COOL-...`), and 5 freshness guarantee claims.

---

2. System Architecture & Course Techniques Covered

The application integrates **6 course capability families** end-to-end:

- **Family 1 — Structured Output & Schema Validation (Lectures 2–3):** FastAPI endpoints validate requests and responses via Pydantic (`ChatRequest` , `ChatResponse`), while `dispatch_tool` defensively parses JSON tool arguments and handles malformed payloads gracefully.
- **Family 2 — Hybrid Retrieval / RAG with Citations (Lectures 6–8):** Combines lexical **Okapi BM25** with a **Thai Subword N-gram & Semantic Concept Vectorizer** fused via **Reciprocal**

Rank Fusion (RRF, `k=60`). Every retrieved chunk injects `[source_file.md # Section]` citations rendered as interactive badges in the UI.

- **Family 3 — Quantitative Evaluation Benchmark (Lecture 9):** A curated **30-test-case benchmark** (`eval/benchmark_data.json`) evaluates retrieval quality (`Recall@1`, `Recall@3`, `Recall@5`, `MRR`) against a naive BM25 baseline and verifies 100% SQL execution accuracy.
- **Family 4 — ReAct Agent Loop with Native Tool Calling (Lectures 10 & 13):** `run_agent_turn` in `src/agent.py` executes a bounded multi-turn loop (`max_steps=5`) over 3 tools (`search_knowledge_base`, `query_database`, `get_order_status`), including a `tool_choice="none"` fallback to guarantee non-empty Thai responses.
- **Family 5 — Guarded Text-to-SQL Execution (Lecture 11):** `guard_sql` in `src/database.py` inspects queries across 5 security layers: strips Markdown code fences, blocks SQL comments (`--`, `/* */`), enforces `SELECT`-only prefix, rejects stacked queries (`;`) and 15 DDL/DML keywords (`DROP`, `DELETE`, `UPDATE`, `INSERT`, etc.), and appends `LIMIT 100`.
- **Family 6 — Served Full-Stack Interface (Lectures 4–5):** FastAPI backend serving a responsive **Thai Retro Neubrutalism Web UI** (`static/index.html`, `static/style.css`, `static/app.js`).

---

3. Quantitative Evaluation Results (30 Benchmark Cases)

The 30-case benchmark (`eval/benchmark_data.json`) is partitioned into:

- **10 RAG Cases (`TC-01` – `TC-10`):** Unstructured questions on Thai fish species, textures, seasons, cooking methods, and Baan Nam Khem sustainable fishing.
- **10 SQL Cases (`TC-11` – `TC-20`):** Inventory counts, pricing, cold-chain order statuses, revenue totals, and freshness claims.
- **5 Compound Multi-Hop Cases (`TC-21` – `TC-25`):** Queries requiring both `search_knowledge_base` and `query_database` / `get_order_status` in a single turn.
- **5 Conversational Chit-Chat Cases (`TC-26` – `TC-30`):** Verifying that the agent responds directly to greetings/thanks without unnecessary tool calls.

Metric	Baseline (BM25 Only)	Improved (Hybrid RAG + RRF)	Gain / Notes
Recall@1 (Top-1 Accuracy)	`80.00%`	**`90.00%`**	**`+12.5%` lift** over BM25 baseline
Recall@3	`100.00%`	**`100.00%`**	`100%` recall within top-3 chunks
Recall@5	`100.00%`	**`100.00%`**	`100%` recall within top-5 chunks
MRR (Mean Reciprocal Rank)	`0.8833`	**`0.9500`**	**`+7.5%` lift** in ranking precision
Guarded SQL Execution Accuracy	N/A	**`100.00%` (10/10)**	All 10 gold queries executed cleanly

4. Failure Case Analysis & Engineering Defenses

- **Failure Case 1 — Unspaced Thai Compound Phrases & Colloquial Synonyms (`TC-03`):**
- *Observation:* When a user asks *"**อยากหาปลาทะเลให้เด็กและคนแก่ทาน ควรเลือกปลาชนิดไหนที่เนื้อนิ่มเคี้ยวง่ายและก้างน้อย**?"*, standard lexical BM25 struggles because Thai words are written without spaces and the user says `"คนแก่"` instead of `"ผู้สูงอายุ"` used in `snapper_and_emperor.md`.
- *Engineering Fix:* Our Hybrid RAG combines Thai character 3/4-gram subword features with domain synonym expansion (`คนแก่` -> `ผู้สูงอายุ` , `ชาวเล` -> `มอแกลน`) and RRF rank fusion, lifting Top-1 Recall from `80.0%` to `90.0%` and MRR from `0.8833` to `0.9500`.
- **Failure Case 2 — Empty Assistant Bubble After Multi-Tool Calls:**
- *Observation:* Occasionally, `gemma-4-E4B-it` completes a tool call turn and returns an empty `msg.content` string on the final turn.
- *Engineering Fix:* Added an explicit post-tool check in `src/agent.py` : if `reply_text` is empty after executing tools, the agent triggers a follow-up synthesis call with `tool_choice="none"` so the user always receives a complete Thai explanation.
- **Failure Case 3 — Adversarial SQL Injection / Destructive Queries:**
- *Observation:* If a user prompts *"**ลบตาราง** orders **ทิ้งเดี๋ยวนี้** (DROP TABLE orders)"*, an unguarded SQL tool could destroy application state.
- *Engineering Fix:* `guard_sql()` deterministically blocks any non- `SELECT` statement, stacked semicolons, or SQL comments before `sqlite3` ever parses the query.